

PROJECT AND TEAM INFORMATION

Project Title

Design and Implementation of a **SKIP LIST**-Based Key-Value Store

Student / Team Information

<i>Team Name:</i>	<i>GEEK VERSE</i>
Team member 1 (Team Lead)	<i>Gera, Sarthak – 2510380077 2510380077@geu.ac.in</i> 
Team member 2	<i>Bhadana, Arjun – 2510370135 2510370135@geu.ac.in</i> 
Team member 3	<i>Gera, Rudraksh – 2510350038 2510350038@geu.ac.in</i> 
Team member 4	<i>Silswal, Divyansh– 2510370989 2510370989@geu.ac.in</i> 

PROPOSAL DESCRIPTION

Motivation

Caches, leaderboards, and autocomplete applications require the data to be sorted and to respond to exact and range queries quickly. Existing data structures are not sufficient to solve this problem. Arrays are fast for searching, but they do not allow fast insertions. Linked lists allow fast insertions but slow searching. Binary search trees allow for both, but when it comes to sorted inputs, they become linear, as they need to traverse through the whole tree to find an element. AVL and red-black trees eliminate this issue, but these trees require complicated rotations that may not be implemented successfully. The problem, therefore, becomes finding a dynamic solution that is efficient while at the same time easy to implement.

State of the Art / Current solution

The most widely accepted approach today will be self-balancing binary search trees — AVL trees or Red-Black trees. In both cases, the running time of each operation is guaranteed to be $O(\log n)$ thanks to rotations. For example, STL in C++ is based on a Red-Black tree (standard library containers utilize these). However, in terms of algorithmic complexity, the logic of rotations is difficult and prone to errors, especially at the undergraduate level. Thus, very few courses actually teach the implementation from the ground up. A lesser-known but proven alternative is the *Skip List* (William Pugh, 1989), used internally in Redis, LevelDB, and Java's `ConcurrentSkipListMap`, but rarely taught.

Project Goals and Milestones

1. Overall Goal: This project aims at developing from scratch a Skip List data structure by means of C++ programming language, as well as implementing a Key-Value store above it as an application. The purpose of this is to prove that randomized data structures can be on par with balanced trees, such as AVL, in terms of their performance properties, while being significantly easier to program correctly if the basic principles of object-oriented programming (the use of templates, encapsulation, constructors/destructors, and operator overloading) are applied in programming.
2. Milestones: The project will run through five main phases pursuant to the review schedule – creating a basic Skip List with necessary operations (Weeks 3–4); generalizing the Skip List using C++ templates along with adequate implementation of memory management (Weeks 5–6); adding a Key-Value Store and implementing the command line interface (Weeks 7–8); testing the performance against `std::map` and `std::unordered_map` (Weeks 9–10); completing documentation, eliminating bugs and preparing a demonstration (Weeks 11–12).

Project Approach

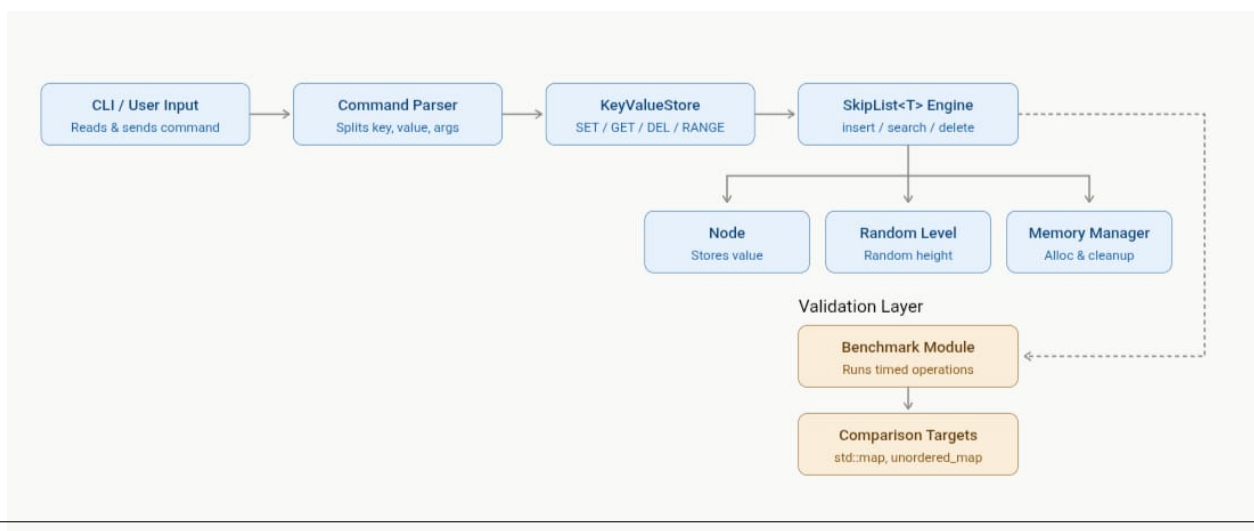
1. Incremental and Test Driven Development: To construct the project, we would be using C++17, VS Code, g++ and Git for version control. The core Skip List will be constructed and tested. Initially just to validate the insert, search and delete operations.
2. Generalization through OOP Principles: After validating the initial structure, we will use the C++ templates to generalize it (SkipList) and make it possible to use multiple types of data. We will utilize encapsulation, constructors/destructors and operator overloading.
3. Application Layer and Empirical Validation: We will, on top of the validated Skip List, build a KeyValueStore class that will implement the SET, GET, DEL and RANGE commands. Finally, we will utilize STL (for example , ,) to build a benchmarking package that will prove or disprove the theoretical performance of the Skip List.

System Architecture (High Level Diagram)

Architecture of the system (High-level diagram)

Layered Design: It uses four-layer architecture. The first layer is CLI/User Input layer that accepts commands. Afterwards, KeyValueStore class processes the commands and then calls SkipList class to make data operations using the actual data stored in each Node and its forward pointers on multiple levels.

Path for parallel validation: At the same time with the main application flow, a separate benchmarking module connects to SkipList class (and STL containers) for independent performance measurements, not affecting the operations of the main application logic.



Project Outcome / Deliverables

This project will result in a fully functional templated Skip List class that supports operations such as insert, search, delete, and display with an expected time complexity of $O(\log n)$. The final outcome of the project will also include a complete and documented source code stored in a version control repository as well as a project report and a presentation that covers everything from the problem description to design choices, to the implementation of the ideas, and to the benchmarking of the results, thus confirming the practical efficiency of the invented Skip List.

Assumptions

This project is designed for single-threaded, in-memory use only; concurrent and distributed access is outside its scope. Input keys are integers or adequately sized strings and do not require a disk in the case of larger amounts of information. The maximum skip list level is assumed to be 16, which is sufficient to cover the expected size of a testing data set that consists of around 10,000-100,000 entries. Standard STL C++17 is supposed to be available in both systems for project development and demonstration.

References

- 1.Pugh, W. (1990). Skip Lists: A Probabilistic Alternative to Balanced Trees. Communications of the ACM, 33(6), 668–676.
- 2.Redis Docs — Sorted Sets: <https://redis.io/docs/latest/develop/data-types/sorted-sets/>
- 3.GeeksforGeeks — Skip List: <https://www.geeksforgeeks.org/dsa/skip-list/>
- 4.<https://en.cppreference.com/w/cpp/container/map>
- 5.LevelDB Documentation — Memtable: <https://github.com/google/leveldb>